

SIH26147 — Automated .IQ / .WAV Signal Analysis Platform

Complete Project Architecture, Requirements & Step-by-Step Implementation Plan

Project objective: Build a realistic software platform that accepts raw IQ/WAV recordings, performs automated digital-signal analysis, extracts signal parameters, visualizes the signal, supports FSK/PSK/QAM demodulation, performs de-interleaving and FEC decoding, and correlates the recovered bit stream.

Important engineering principle: Treat the system as a DSP/communications platform with a web interface—not as a normal CRUD web application. The signal-processing pipeline is the core of the project.

1. Problem Understanding

Signals captured from terrestrial sources may be recorded in HF, VHF and UHF ranges and stored as .IQ or .WAV data. Because recordings can come from different sensors and locations, metadata and acquisition parameters may be incomplete. Manual inspection makes fine-grained parameter extraction difficult. The proposed system automates as much of this analysis as possible and reports estimated values together with confidence and provenance.

Target outputs include sampling frequency, carrier frequency, bandwidth, symbol rate, modulation, SNR, candidate FEC/interleaving configuration, demodulated symbols/bits, visual plots and bit-stream correlation results.

2. Complete High-Level Architecture

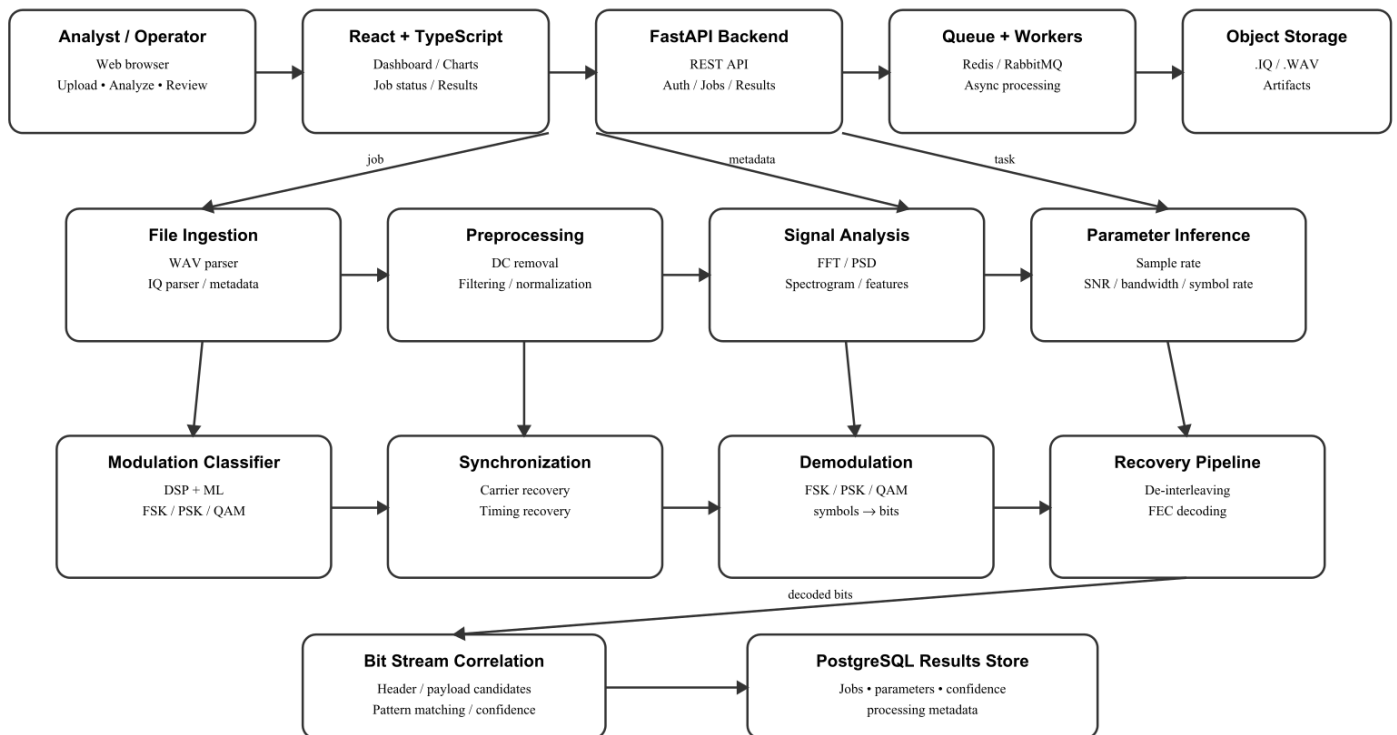


Figure 1. End-to-end platform architecture. The GUI/API layer is separated from asynchronous signal-processing workers so large recordings do not block the web application.

Core layers

- Presentation: React + TypeScript dashboard.
- API/control plane: FastAPI for authentication, uploads, job creation and results.
- Processing plane: Python, NumPy/SciPy and GNU Radio, with C++ only for components where profiling shows a real performance need.
- Storage: object storage for raw recordings/artifacts and PostgreSQL for metadata/results.
- Queue: Redis or RabbitMQ for asynchronous jobs.
- Deployment: Docker containers, Linux server/cloud VM; Kubernetes is optional rather than a first milestone.

3. Detailed Signal Processing Pipeline

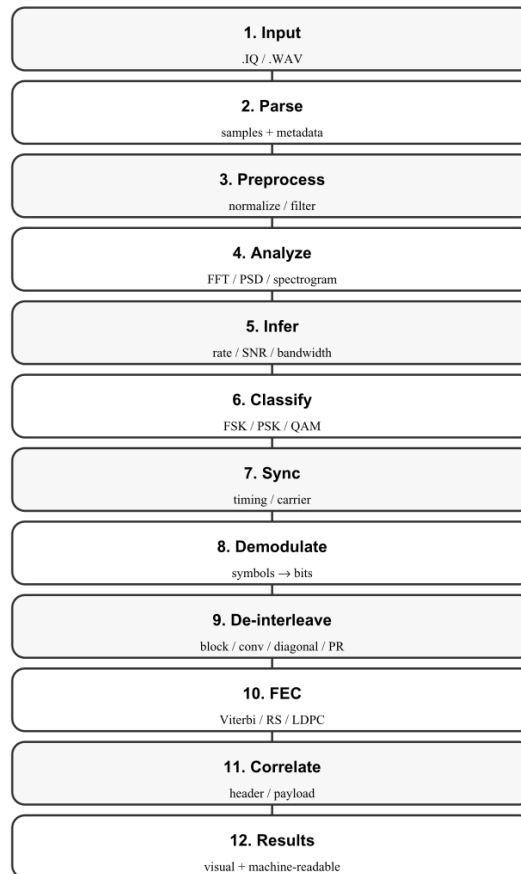


Figure 2. Recommended processing sequence. Each stage should expose inputs, outputs, configuration and confidence/quality metrics.

A key design rule is that uncertain parameters must remain uncertain. For example, if a sample rate is read from reliable file metadata, label it as metadata-derived. If it is inferred, label it as estimated and provide a confidence/range rather than pretending it is exact.

4. Functional Requirements

ID	Requirement
FR-01	Upload and validate .IQ and .WAV recordings.
FR-02	Parse available metadata: sample rate, channels, format, center frequency where present.
FR-03	Convert input to an internal numerical representation suitable for DSP.
FR-04	Preprocess: DC removal, amplitude normalization, filtering and optional resampling.
FR-05	Generate waveform, FFT/PSD, spectrogram/waterfall and constellation visualizations.
FR-06	Estimate signal parameters: sample rate, carrier frequency, bandwidth, symbol rate, SNR and related features.
FR-07	Classify candidate modulation types including FSK, PSK and QAM families.
FR-08	Demodulate supported FSK, PSK and QAM signals.
FR-09	Perform timing and carrier synchronization required for reliable demodulation.
FR-10	Support block, convolutional, diagonal and pseudo-random de-interleaving where configuration is known/inferred.
FR-11	Support convolutional/Viterbi, Reed-Solomon, concatenated and LDPC decoding as staged capabilities.
FR-12	Correlate recovered bit streams and identify candidate headers/payload boundaries.
FR-13	Display confidence, provenance, processing stage and error information.
FR-14	Run large analyses asynchronously and expose progress/status to the GUI.
FR-15	Export results as JSON/CSV and a human-readable analysis report.

5. Non-Functional Requirements

- Reliability: failed jobs must be recoverable/retryable without corrupting source files.
- Scalability: processing workers should be horizontally scalable for independent recordings.
- Observability: structured logs, processing durations, stage-level errors and health checks.
- Reproducibility: store algorithm/model version, configuration and input hash with every result.
- Security: authenticated analyst access, role-based authorization, HTTPS, safe file validation and isolation of uploaded files.
- Performance: stream/chunk large recordings instead of loading unnecessarily large arrays into memory.
- Explainability: show why a parameter was selected—metadata, DSP estimate, ML prediction, or combination.

6. Module Architecture

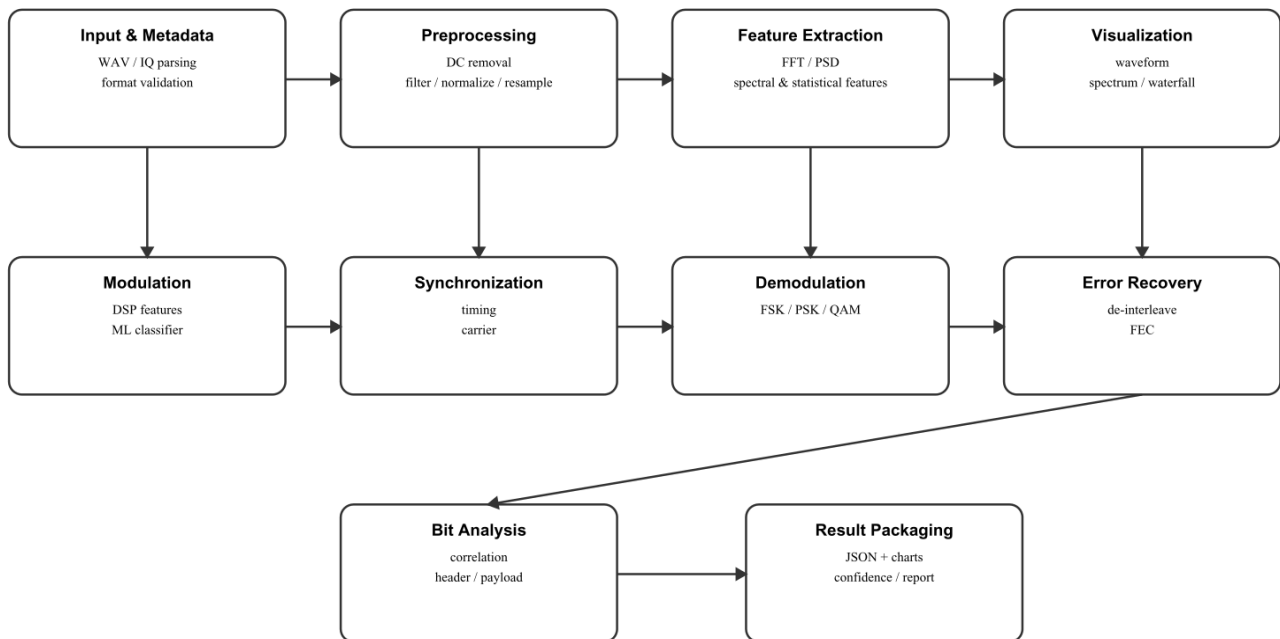


Figure 3. Functional module breakdown.

7. Data Model

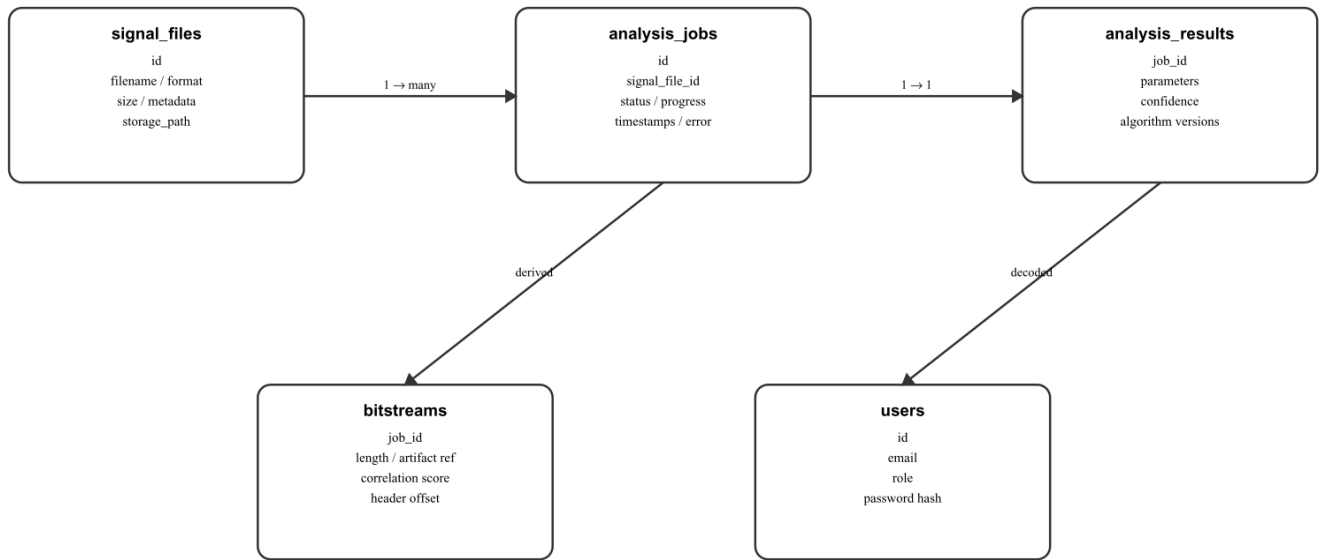


Figure 4. Minimal production-oriented data model. Raw signal arrays should live in object storage rather than in relational database rows.

Suggested entities

users: id, email, password_hash, role, created_at.

signal_files: id, filename, format, size, sample_rate, channels, sample_format, storage_path, checksum, uploaded_at.

analysis_jobs: id, signal_file_id, status, progress, pipeline_config, started_at, completed_at, error.

analysis_results: job_id, estimated parameters, modulation candidates, confidence, algorithm/model versions.

bitstreams: job_id, length, artifact reference, correlation score, header offsets.

8. Deployment Architecture

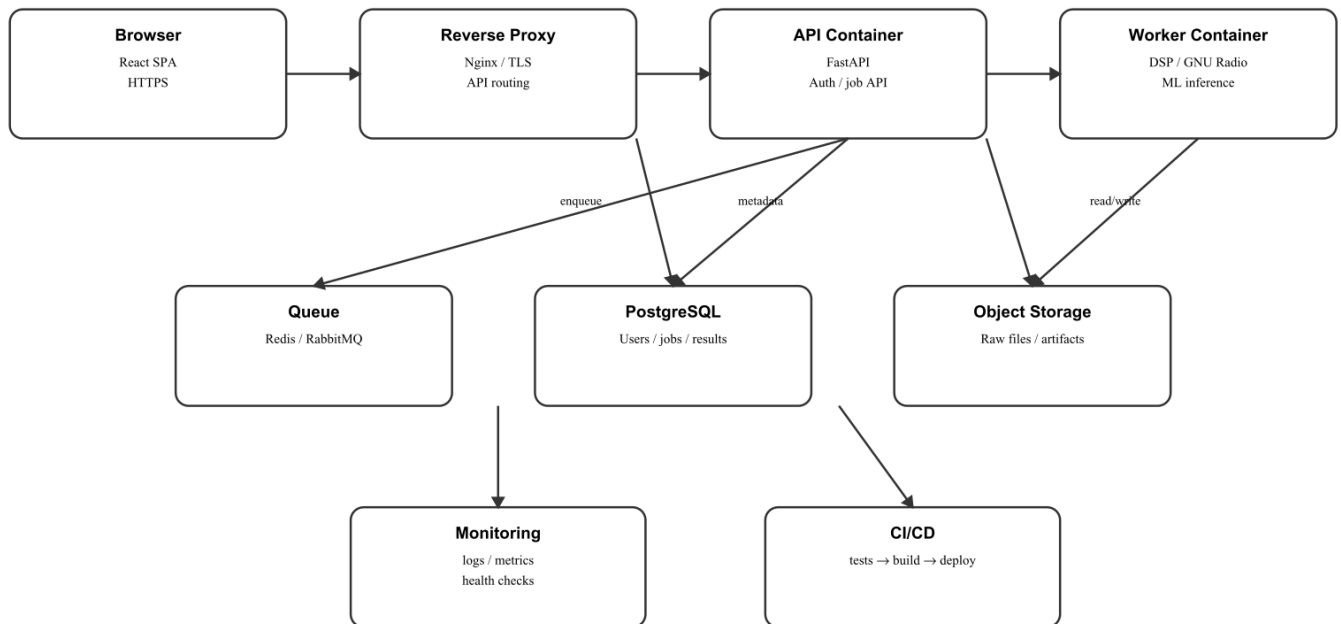


Figure 5. Realistic container/cloud deployment. Start with Docker Compose or a small VM; introduce Kubernetes only when workload and operational needs justify it.

9. Technology Stack

Layer	Recommended technology	Purpose
Frontend	React + TypeScript + Vite	Analyst GUI and interactive visualizations
Charts	Plotly / WebGL-capable plotting	Waveform, spectrum, waterfall, constellation
API	Python + FastAPI	REST API, auth, jobs, results
DSP	NumPy + SciPy + GNU Radio	Signal processing and communications blocks
ML	scikit-learn / PyTorch	Modulation classification and selected inference tasks
Performance	C++ (optional)	Only for profiled bottlenecks / specialized DSP
Queue	Redis or RabbitMQ	Asynchronous job execution
Database	PostgreSQL	Users, jobs, metadata and analysis results
Object storage	S3-compatible storage	Large .IQ/.WAV files and generated artifacts
Packaging	Docker	Repeatable local/demo/cloud deployment
Observability	Structured logs + metrics	Debugging, health and performance tracking

10. Suggested Repository Structure

```
signal-analysis-platform/  
■ ■ ■ frontend/  
■ ■ ■ ■ src/components/ src/pages/ src/charts/ src/services/  
■ ■ ■ backend/  
■ ■ ■ ■ app/api/ app/models/ app/schemas/ app/services/  
■ ■ ■ processing/  
■ ■ ■ ■ io/ preprocessing/ analysis/ synchronization/  
■ ■ ■ ■ modulation/ demodulation/ deinterleaving/ fec/ correlation/  
■ ■ ■ ml/  
■ ■ ■ ■ datasets/ training/ models/ evaluation/  
■ ■ ■ workers/  
■ ■ ■ tests/  
■ ■ ■ docker/  
■ ■ ■ docs/  
■ ■ ■ README.md
```

11. Step-by-Step Development Plan

Phase 0 — Learn the domain: I/Q, sampling, Nyquist, FFT, spectrum, SNR, modulation, symbol rate.

Phase 1 — Signal laboratory: Generate known BPSK/QPSK/FSK/QAM signals, add noise, save IQ and recover them.

Phase 2 — Input layer: Implement WAV/IQ parsing, metadata extraction, validation and checksums.

Phase 3 — DSP visualization: Waveform, FFT/PSD, spectrogram/waterfall and constellation.

Phase 4 — Parameter extraction: SNR, bandwidth, carrier frequency and symbol-rate estimation; preserve uncertainty.

Phase 5 — Modulation detection: Build classical baselines first, then train an ML classifier and compare results.

Phase 6 — Synchronization + demodulation: Timing/carrier recovery followed by FSK/PSK/QAM demodulation.

Phase 7 — De-interleaving: Implement and unit-test block, convolutional, diagonal and pseudo-random methods.

Phase 8 — FEC: Viterbi first, then Reed-Solomon, concatenated codes and LDPC as staged modules.

Phase 9 — Bit analysis: Correlation, known-pattern search and candidate header/payload segmentation.

Phase 10 — Backend integration: FastAPI + queue + worker + PostgreSQL + object storage.

Phase 11 — Frontend integration: Upload, job progress, analysis tabs, plots, parameters and export.

Phase 12 — Testing + deployment: Golden test vectors, noise tests, Docker, CI/CD, security and performance testing.

12. MVP Scope for SIH

A realistic first demonstrator should not attempt every FEC/interleaver variant at once. The recommended MVP is: .IQ/.WAV upload → preprocessing → waveform/FFT/waterfall/constellation → parameter extraction → modulation classification → synchronization → FSK/PSK/QAM demodulation → recovered bit stream → basic correlation.

After the MVP works on controlled test signals, add de-interleaving and FEC. This order lets the team demonstrate a functioning end-to-end system early and reduces the risk of spending the whole project on one difficult decoder.

13. AI/ML Strategy

Use AI selectively. A strong design is a hybrid pipeline: DSP extracts robust signal features; an ML classifier proposes modulation candidates; a validation layer checks physical consistency; the final result includes confidence and provenance. Unknown/ambiguous signals should be allowed as an output rather than forcing a wrong class.

14. Testing Strategy

Create a controlled golden dataset with known modulation, symbol rate, SNR, coding and interleaving settings. For each test, retain the original transmitted bits so recovered BER can be calculated. Test across different noise levels, frequency offsets, timing offsets, amplitude scaling and file formats. Every DSP module should have unit tests; every end-to-end pipeline should have regression vectors.

Key metrics

- Modulation classification accuracy and confusion matrix.
- Parameter estimation error (sample rate, carrier, bandwidth, symbol rate).
- BER before and after FEC.
- Demodulation success rate under different SNRs.
- Processing throughput and peak memory.
- Job failure/retry rate.

15. Final SIH Demonstration Flow

1. Analyst uploads an unknown .IQ or .WAV file.
2. System validates the file and creates an analysis job.
3. Dashboard shows live processing stages.
4. System displays waveform, spectrum and waterfall.
5. Parameter panel reports estimated values with confidence/provenance.
6. Modulation classifier presents candidate classes.
7. Supported demodulator runs automatically or after analyst confirmation.
8. De-interleaver/FEC stages run when their configuration is available with sufficient confidence.
9. Recovered bit stream is displayed and correlated against reference patterns.
10. Analyst exports the complete result package.

16. Practical Team Split

Member A — DSP/communications: signal theory, preprocessing, synchronization, demodulation.

Member B — ML: dataset generation, feature engineering, classifier and evaluation.

Member C — backend/integration: FastAPI, queue, workers, database, object storage and APIs.

Member D — frontend/DevOps: React dashboard, plots, Docker, CI/CD and deployment.

17. Critical Risks & Mitigations

Risk: unknown recordings lack metadata. Mitigation: distinguish metadata-derived values from estimates and show confidence.

Risk: FEC/interleaving cannot be uniquely inferred from arbitrary data. Mitigation: support candidate configurations, known test vectors and explicit 'unknown' outcomes.

Risk: large IQ files exhaust memory. Mitigation: chunked/streaming processing and object storage.

Risk: ML works only on synthetic signals. Mitigation: test on varied datasets and clearly report domain shift limitations.

Risk: trying to implement every decoder first. Mitigation: deliver MVP first, then add FEC/interleaving modules incrementally.